SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA05**

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.* (BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

2. OCR Pipeline for Document Processing

A document processing system must:

A. Read input images containing printed text
B. Enhance image quality (noise removal, contrast improvement)
C. Segment text regions
D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Answer:

```
FUNCTION process_single_document(image_path):
   # 1. Preprocessing Stage (Noise reduction & contrast enhancement)
   img = cv2.imread(image_path)
   IF img IS NULL: RETURN NULL

   gray = cv2.cvtColor(img, COLOR_BGR2GRAY)
   denoised = cv2.fastNlMeansDenoising(gray, h=10) # Remove noise while
preserving edges
```

```
    # Adaptive thresholding handles non-uniform illumination/contrast
    enhanced = cv2.adaptiveThreshold(denoised, 255,
ADAPTIVE_THRESH_GAUSSIAN_C,
                                     THRESH_BINARY, 11, 2)

    # 2. Segmentation Stage (Isolating text lines/regions)
    kernel = cv2.getStructuringElement(MORPH_RECT, (15, 3)) # Horizontal dilation
for line grouping
    morphed = cv2.morphologyEx(enhanced, MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(morphed, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE)

    # Filter contours by aspect ratio and minimum area to remove artifacts
    text_regions = []
    FOR EACH cnt IN contours:
        x, y, w, h = cv2.boundingRect(cnt)
        IF w > 20 AND h > 10 AND (w / h) > 0.5:
            text_regions.APPEND(img[y:y+h, x:x+w])
            cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # 3. Extraction Stage (OCR Integration)
    extracted_text = ""
    FOR EACH region IN text_regions:
        text = pytesseract.image_to_string(region, config='--psm 6')
        extracted_text += text + "\n"

    RETURN extracted_text, img

FUNCTION batch_ocr_pipeline(image_folder_path):
    image_files = get_all_images(image_folder_path)

    FOR EACH file IN image_files:
        text, annotated_img = process_single_document(file)
        IF text IS NOT NULL:
            DISPLAY annotated_img
            SAVE text TO file + "_ocr.txt"
```

3. Motion Detection and Tracking System

A surveillance system must:

A. Read video input
B. Detect motion between consecutive frames

C. Track moving objects
D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Answer:

```
FUNCTION run_motion_detection(video_source):
    cap = cv2.VideoCapture(video_source)

    # Dynamic Background Subtractor handles dynamic lighting & swaying objects
    bg_subtractor = cv2.createBackgroundSubtractorMOG2(history=500,
varThreshold=16, detectShadows=True)
    tracker_dict = MULTI_TRACKER_INIT()

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        # Preprocessing: Reduce noise to prevent false motion triggers
        blurred = cv2.GaussianBlur(frame, (21, 21), 0)

        # Frame Differencing via Background Subtraction
        fg_mask = bg_subtractor.apply(blurred)
        _, thresh = cv2.threshold(fg_mask, 250, 255, THRESH_BINARY) # Remove
shadows

        # Morphological operations to merge motion fragments
        kernel = cv2.getStructuringElement(MORPH_ELLIPSE, (5, 5))
        cleaned_mask = cv2.morphologyEx(thresh, MORPH_OPEN, kernel)
        cleaned_mask = cv2.dilate(cleaned_mask, kernel, iterations=2)

        # Detection & Tracking
        contours, _ = cv2.findContours(cleaned_mask, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE)

        FOR EACH cnt IN contours:
            IF cv2.contourArea(cnt) < 1000: CONTINUE # Filter small dynamic noise

            x, y, w, h = cv2.boundingRect(cnt)
```

```
            # Update/Initialize centroid tracker
            centroid = (x + w//2, y + h//2)
            tracker_dict.update_or_add(centroid, (x, y, w, h))

            # Visualization
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.circle(frame, centroid, 4, (0, 0, 255), -1)

        cv2.imshow("Surveillance Feed", frame)
        IF cv2.waitKey(1) == ORD('q'): BREAK

    cap.release()
    cv2.destroyAllWindows()
```

4. Vehicle Counting System using Video

A traffic monitoring system must:

A. Read video frames
B. Detect vehicles entering a region of interest (ROI)
C. Track movement across frames
D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

Answer:

```
FUNCTION vehicle_counting_pipeline(video_path, roi_line_y):
    cap = cv2.VideoCapture(video_path)
    detector = cv2.createBackgroundSubtractorMOG2()

    trackers = MAP() # Stores object_id -> (prev_y, current_y)
    vehicle_counter = 0
    next_object_id = 0

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        # ROI Isolation
        mask = detector.apply(frame)
        _, thresh = cv2.threshold(mask, 200, 255, THRESH_BINARY)
```

```
    contours, _ = cv2.findContours(thresh, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE)
    current_frame_centroids = []

    FOR EACH cnt IN contours:
      IF cv2.contourArea(cnt) > 1500: # Vehicle size threshold
        x, y, w, h = cv2.boundingRect(cnt)
        cx, cy = x + w//2, y + h//2
        current_frame_centroids.APPEND((cx, cy, x, y, w, h))

    # Matching Centroids (Nearest Neighbor tracking)
    updated_trackers = MAP()
    FOR EACH (cx, cy, x, y, w, h) IN current_frame_centroids:
      matched_id = find_closest_id(trackers, (cx, cy), max_dist=50)

      IF matched_id IS NOT NULL:
        prev_y = trackers[matched_id].y
        updated_trackers[matched_id] = (cx, cy)

        # Double-counting avoidance: Trigger count only when crossing ROI
threshold line directionally
        IF prev_y < roi_line_y AND cy >= roi_line_y:
          vehicle_counter += 1
      ELSE:
        updated_trackers[next_object_id] = (cx, cy)
        next_object_id += 1

    trackers = updated_trackers

    # Visualization
    cv2.line(frame, (0, roi_line_y), (frame.width, roi_line_y), (0, 0, 255), 2) # ROI
Line
    cv2.putText(frame, "Count: " + STR(vehicle_counter), (50, 50),
FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.imshow("Traffic Monitor", frame)

    IF cv2.waitKey(30) == ORD('q'): BREAK

  cap.release()
```

6. Real-Time Object Detection Pipeline

A system must:

A. Capture video frames

B. Detect objects using a trained model
C. Draw bounding boxes and labels
D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

Answer:

```
FUNCTION run_realtime_detection():

    # Load lightweight deep learning model optimized for real-time inference

    net = cv2.dnn.readNetFromONNX("yolov8n.onnx")

    net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)

    net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU) # Or
DNN_TARGET_CUDA for GPU



    cap = cv2.VideoCapture(0) # Live camera input



    WHILE cap.isOpened():

        ret, frame = cap.read()

        IF NOT ret: BREAK



        # Preprocessing: Convert image to normalized blob

        blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True,
crop=False)

        net.setInput(blob)



        # Forward Pass

        outputs = net.forward()
```

```
    boxes, confidences, class_ids = parse_detections(outputs,
confidence_threshold=0.5)



    # Non-Maximum Suppression (NMS) to eliminate duplicate overlapping boxes

    indices = cv2.dnn.NMSBoxes(boxes, confidences, score_threshold=0.5,
nms_threshold=0.4)



    # Visualization

    FOR EACH i IN indices:

        box = boxes[i]

        x, y, w, h = box[0], box[1], box[2], box[3]

        label = CLASSES[class_ids[i]] + ": " + STR(ROUND(confidences[i], 2))



        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)

        cv2.putText(frame, label, (x, y - 10), FONT_HERSHEY_SIMPLEX, 0.5, (255,
0, 0), 2)



    cv2.imshow("Real-Time Object Detector", frame)

    IF cv2.waitKey(1) == ORD('q'): BREAK



cap.release()

cv2.destroyAllWindows()
```

7. Image Difference Detection System

A system must:

A. Read two images
B. Compare them
C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

Answer:

```
FUNCTION detect_image_differences(img_path1, img_path2):
    img1 = cv2.imread(img_path1)
    img2 = cv2.imread(img_path2)

    # 1. Structural Preprocessing & Alignment Check
    IF img1.shape != img2.shape:
        img2 = cv2.resize(img2, (img1.width, img1.height))

    gray1 = cv2.cvtColor(img1, COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, COLOR_BGR2GRAY)

    # Apply Gaussian Blur to filter camera sensor noise/minor pixel shifts
    blur1 = cv2.GaussianBlur(gray1, (5, 5), 0)
    blur2 = cv2.GaussianBlur(gray2, (5, 5), 0)

    # 2. Absolute Difference Computation
    diff = cv2.absdiff(blur1, blur2)

    # 3. Thresholding & Morphological Noise Filtering
    _, thresh = cv2.threshold(diff, 30, 255, THRESH_BINARY)
    kernel = cv2.getStructuringElement(MORPH_RECT, (3, 3))
    cleaned_diff = cv2.morphologyEx(thresh, MORPH_OPEN, kernel) # Remove speckle noise

    # 4. Contour Detection and Visualization
```

```
    contours, _ = cv2.findContours(cleaned_diff, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE)


    output = img2.copy()

    FOR EACH cnt IN contours:

        IF cv2.contourArea(cnt) > 50: # Ignore negligible pixel variances

            x, y, w, h = cv2.boundingRect(cnt)

            cv2.rectangle(output, (x, y), (x + w, y + h), (0, 0, 255), 2)


    cv2.imshow("Image Differences", output)

    cv2.waitKey(0)
```

12. Face Detection with ROI Extraction

A system must:

A. Detect faces in an image
B. Extract face regions
C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

Answer:

```
FUNCTION detect_and_save_faces(image_path, output_dir):

    img = cv2.imread(image_path)

    gray = cv2.cvtColor(img, COLOR_BGR2GRAY)


    # Use Haar Cascade or DNN Face Detector

    face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")


    # ScaleFactor and minNeighbors control detection robustness and redundancy
```

```
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    face_count = 0
    FOR EACH (x, y, w, h) IN faces:
        # Extract Region of Interest (ROI)
        face_roi = img[y:y+h, x:x+w]

        # Save cropped faces to filesystem with unique identifiers
        save_path = output_dir + "/face_" + STR(face_count) + ".jpg"
        cv2.imwrite(save_path, face_roi)

        # Draw bounding boxes on primary visualization frame
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
        face_count += 1

    RETURN img, face_count
```

13. Video Stabilization Pipeline

A system must:

A. Read video frames
B. Detect motion between frames
C. Stabilize output

Constraints:

- Maintain real-time processing

- Reduce jitter

Write pseudocode including motion estimation, transformation, and display.

Answer:

```
FUNCTION stabilize_video(video_path):
    cap = cv2.VideoCapture(video_path)

    ret, prev_frame = cap.read()
    IF NOT ret: RETURN
    prev_gray = cv2.cvtColor(prev_frame, COLOR_BGR2GRAY)

    transforms = []

    # Step 1: Optical Flow Tracking across frame sequence
    WHILE cap.isOpened():
        ret, curr_frame = cap.read()
        IF NOT ret: BREAK

        curr_gray = cv2.cvtColor(curr_frame, COLOR_BGR2GRAY)

        # Detect tracking points in previous frame
        prev_pts = cv2.goodFeaturesToTrack(prev_gray, maxCorners=200,
qualityLevel=0.01, minDistance=30)

        # Calculate optical flow (Lucas-Kanade)
        curr_pts, status, _ = cv2.calcOpticalFlowPyrLK(prev_gray, curr_gray, prev_pts,
NULL)

        # Filter valid points
```

```
        idx = WHERE(status == 1)
        prev_valid = prev_pts[idx]
        curr_valid = curr_pts[idx]

        # Estimate rigid transformation matrix (translation + rotation)
        m, _ = cv2.estimateAffinePartial2D(prev_valid, curr_valid)

        dx = m[0, 2]
        dy = m[1, 2]
        da = ATAN2(m[1, 0], m[0, 0])

        transforms.APPEND([dx, dy, da])
        prev_gray = curr_gray

    # Step 2: Smooth Motion Trajectory (Moving Average Filter to remove high-frequency jitter)
    smoothed_transforms = apply_moving_average_filter(transforms, radius=30)

    # Step 3: Apply smoothed transformations
    cap.set(CAP_PROP_POS_FRAMES, 0) # Reset stream
    FOR EACH [dx, dy, da] IN smoothed_transforms:
        ret, frame = cap.read()

        # Reconstruct affine matrix
        m_smooth = MATRIX([[COS(da), -SIN(da), dx], [SIN(da), COS(da), dy]])

        # Warp current frame to stabilized trajectory
```

```
        stabilized_frame = cv2.warpAffine(frame, m_smooth, (frame.width,
frame.height))



        cv2.imshow("Stabilized Output", stabilized_frame)

        cv2.waitKey(10)



    cap.release()
```

14. Text Detection in Natural Scene Images

A system must:

A. Read images
B. Detect text regions
C. Highlight detected areas

Constraints:

- Handle varying backgrounds
- Improve detection accuracy

Write pseudocode including preprocessing, region detection, and visualization.

Answer:

```
FUNCTION detect_scene_text(image_path):
    img = cv2.imread(image_path)
    h, w, _ = img.shape

    # Preprocessing: Multi-scale processing to handle complex natural light
    gray = cv2.cvtColor(img, COLOR_BGR2GRAY)

    # Load EAST Text Detector (DNN optimized for arbitrary orientation & natural
scenes)
    net = cv2.dnn.readNet("frozen_east_text_detection.pb")

    # Resize image to multiples of 32 required by EAST architecture
    new_w, new_h = (320, 320)
    r_w, r_h = w / FLOAT(new_w), h / FLOAT(new_h)
    blob = cv2.dnn.blobFromImage(img, 1.0, (new_w, new_h), (123.68, 116.78,
103.94), swapRB=True, crop=False)

    net.setInput(blob)
    layer_names = ["feature_fusion/Conv_7/Sigmoid", "feature_fusion/concat_3"]
```

```
    scores, geometry = net.forward(layer_names)

    # Decode bounding boxes from geometry and confidence maps
    boxes, confidences = decode_east_predictions(scores, geometry,
min_confidence=0.5)

    # Suppress overlapping detection boxes
    indices = cv2.dnn.NMSBoxesRotated(boxes, confidences, score_threshold=0.5,
nms_threshold=0.4)

    # Draw detections on original image space
    FOR EACH i IN indices:
        box = boxes[i]
        # Rescale bounding box coordinates back to original image dimensions
        points = cv2.boxPoints(box)
        points = scale_points(points, r_w, r_h)

        cv2.polylines(img, [INT_32(points)], isClosed=True, color=(0, 255, 0),
thickness=2)

    cv2.imshow("Scene Text Detection", img)
    cv2.waitKey(0)
```

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient